```
import numpy as np
import pandas as pd

STATES = ['S0', 'S1', 'G']
NON_TERMINAL_STATES = ['S0', 'S1']
ACTIONS = ['Up', 'Down', 'Left', 'Right']

GAMMA = 0.9
GOAL_REWARD = 100.0
NORMAL_REWARD = -1.0
WALL_REWARD = -1.0
THETA = 1e-6

def step(state, action):
    if state == 'G':
        return 'G', 0.0

    if state == 'S0':
        if action == 'Down':
            return 'S1', NORMAL_REWARD
        elif action in ['Up', 'Left', 'Right']:
            return 'S0', WALL_REWARD

    elif state == 'S1':
        if action == 'Right':
            return 'G', GOAL_REWARD
        elif action == 'Up':
            return 'S0', NORMAL_REWARD
        elif action in ['Down', 'Left']:
            return 'S1', WALL_REWARD

    return state, NORMAL_REWARD

print("================================================")
print("             MDP ENVIRONMENT DETAILS             ")
print("================================================")
print(f"States: {STATES}")
print(f"Non-Terminal States: {NON_TERMINAL_STATES}")
print(f"Actions: {ACTIONS}")
print(f"Discount Factor (gamma): {GAMMA}")
print(f"Goal Reward: {GOAL_REWARD}")
print(f"Normal Step / Wall Reward: {NORMAL_REWARD}")
print("\nTransition Information:")

for s in NON_TERMINAL_STATES:
    for a in ACTIONS:
        next_s, r = step(s, a)
        print(f"  State: {s:2s} | Action: {a:5s} -> Next State: {next_s:2s} | Reward: {r:5.1f}")
print("================================================\n")
```

```
V = {s: 0.0 for s in STATES}
policy = {'S0': 'Up', 'S1': 'Up'}

print(f"Initial Policy: {policy}")
print(f"Initial Values: {V}\n")

policy_stable = False
outer_iteration = 0

print("================================================")
print("           STARTING POLICY ITERATION            ")
print("================================================")

while not policy_stable:
    outer_iteration += 1
    print(f"\n--- POLICY ITERATION {outer_iteration} ---")
    print(f"Current Policy: {policy}")

    eval_iter = 0
    while True:
        eval_iter += 1
        delta = 0.0
        for s in NON_TERMINAL_STATES:
            v_old = V[s]
            action = policy[s]
            next_s, reward = step(s, action)
            V[s] = reward + GAMMA * V[next_s]
            delta = max(delta, abs(v_old - V[s]))

        if delta < THETA:
            break

    print(f"Policy Evaluation converged in {eval_iter} iterations.")
    print("State Values after Evaluation:")
    for s in STATES:
        print(f"  V({s}) = {V[s]:.4f}")

    policy_stable = True
    print("\nPolicy Improvement Step:")

    for s in NON_TERMINAL_STATES:
        old_action = policy[s]

        q_values = {}
        for a in ACTIONS:
            next_s, reward = step(s, a)
            q_values[a] = reward + GAMMA * V[next_s]

        best_action = max(q_values, key=q_values.get)
        policy[s] = best_action
```

```
        print(f"  State {s}: Best Action = {best_action} (Q-values: {q_values})")

        if old_action != best_action:
            policy_stable = False

print("\n================================================")
print("             FINAL OPTIMAL RESULTS               ")
print("================================================")

q_table = {s: {} for s in NON_TERMINAL_STATES}
for s in NON_TERMINAL_STATES:
    for a in ACTIONS:
        next_s, reward = step(s, a)
        q_table[s][a] = reward + GAMMA * V[next_s]

q_df = pd.DataFrame(q_table).T
print("\nFinal Action-Value Table Q*(s, a):")
print(q_df.round(2))

results = []
for s in STATES:
    if s == 'G':
        results.append({
            'State': s,
            'Optimal Value V*(s)': V[s],
            'Optimal Action': 'Terminal'
        })
    else:
        results.append({
            'State': s,
            'Optimal Value V*(s)': round(V[s]),
            'Optimal Action': policy[s]
        })

results_df = pd.DataFrame(results)
print("\nOptimal State Values and Policy:")
print(results_df.to_string(index=False))

print("\n================================================")
print("        BELLMAN OPTIMALITY VERIFICATION          ")
print("================================================")
verification_passed = True
for s in NON_TERMINAL_STATES:
    max_q = max(q_table[s].values())
    v_val = V[s]
    diff = abs(v_val - max_q)
    print(f"State {s:2s}: V*(s) = {v_val:8.4f} | max_a Q*(s, a) = {max_q:8.4f} | Diff = {diff:.6f}")
    if diff > THETA:
        verification_passed = False

if verification_passed:
```

```
    print("\nVerification Status: PASSED (V*(s) == max_a Q*(s, a) for all non-terminal states)")
else:
    print("\nVerification Status: FAILED")
```

```
--- POLICY ITERATION 1 ---
Current Policy: {'S0': 'Up', 'S1': 'Up'}
Policy Evaluation converged in 133 iterations.
State Values after Evaluation:
  V(S0) = -10.0000
  V(S1) = -10.0000
  V(G) = 0.0000

Policy Improvement Step:
  State S0: Best Action = Up (Q-values: {'Up': -9.99999261252091, 'Down': -9.999993351268818, 'Left': -9.99999261252091, 'Right': -9.99999261252091})
  State S1: Best Action = Right (Q-values: {'Up': -9.99999261252091, 'Down': -9.999993351268818, 'Left': -9.999993351268818, 'Right': 100.0})

--- POLICY ITERATION 2 ---
Current Policy: {'S0': 'Up', 'S1': 'Right'}
Policy Evaluation converged in 2 iterations.
State Values after Evaluation:
  V(S0) = -10.0000
  V(S1) = 100.0000
  V(G) = 0.0000

Policy Improvement Step:
  State S0: Best Action = Down (Q-values: {'Up': -9.999994016141937, 'Down': 89.0, 'Left': -9.999994016141937, 'Right': -9.999994016141937})
  State S1: Best Action = Right (Q-values: {'Up': -9.999994016141937, 'Down': 89.0, 'Left': 89.0, 'Right': 100.0})

--- POLICY ITERATION 3 ---
Current Policy: {'S0': 'Down', 'S1': 'Right'}
Policy Evaluation converged in 2 iterations.
State Values after Evaluation:
  V(S0) = 89.0000
```

```
Verification Status: PASSED (V*(s) == max_a Q*(s, a) for all non-terminal states)
```